Developer Community SASTRA

Google Developer Groups
On Campus • SASTRA Deemed University

DCS - Web Cluster
Backend Task

Google form link - Click here

Submission Deadline: 05th October, 2026

Task: Develop a Backend Service for an Event Ticketing Platform

Background:

As a backend developer joining a fast-growing event ticketing startup, you have been asked to design and build the API that powers event discovery, ticket booking, and the organizer dashboard. Real event platforms have to deal with problems most tutorials skip — hundreds of people racing to book the last few seats, events that can't be modified once they've started, refunds when things go wrong, and tight authorization rules so an organizer can never touch someone else's event. This task will walk you through building exactly that kind of service.

Requirements:

Your job is to build a backend service that lets attendees browse and book tickets for upcoming events (concerts, workshops, meetups, conferences) and lets organizers create and manage their own events. The service should expose a RESTful API, validate every request, persist data in a relational database, and enforce role-based rules end-to-end.

In short: an attendee opens the app, filters events by city, date, or category, picks one, and books a few tickets. An organizer logs into their dashboard, creates a new event, watches tickets sell, and can view every booking against their events.

All endpoints for the basic CRUD operations on the users, events, and bookings tables should be available with proper privileged authentication. JWT-based authentication is preferred with two kinds of users: Attendee, Organizer.

Attendees should be able to: browse and filter events (by category, city, date range, price), view event details and remaining capacity, book one or more tickets for an event, view their booking history, and cancel a booking within the allowed cancellation window.

Organizers should be able to: create, edit, and delete their own events, view all bookings made against their events, see a sales summary per event (tickets sold, revenue, remaining capacity), and cancel an event (which should cascade a refund status to every affected booking).

The Registration & Auth service should:

- Register a new user with a chosen role (Attendee or Organizer).
- Authenticate users and issue access tokens (refresh tokens are a plus).
- Validate every incoming payload and reject malformed or incomplete requests.
- Respond with a clear confirmation or a structured error message.

The Events & Booking service should:

- Authenticate the caller and enforce role-based authorization on every endpoint.
- Prevent overbooking — even under concurrent requests for the last few seats.
- Reject bookings for events that have already started or been cancelled.
- Only let an organizer read or modify events they own.
- Support pagination and filtering on list endpoints so responses stay small.
- Respond with structured JSON on both success and failure.

Database models and their relations should be established accordingly (a user has many bookings, an event has many bookings, an organizer has many events, and so on). Think carefully about which fields need indexes and which relationships need cascade rules.

The responses should be in JSON format and the API should be RESTful. Extra points will be allotted for following OpenAPI guidelines and shipping a live Swagger / Redoc page.

Bonus challenges (extra credit)

These are optional, but each one teaches a concept that comes up in almost every real backend job. Attempt whatever you find interesting — quality of one is worth more than a shallow attempt at all.

- Rate-limit the booking endpoint per user (for example, 5 booking attempts per minute) using a middleware or a Redis-backed limiter.
- Send a confirmation email on successful booking (a mock SMTP catcher like Mailhog is fine — no real emails needed).
- Add a background job that flips events to a "completed" status once their datetime has passed.
- Cache the public events list in Redis with sensible invalidation when an organizer updates an event.
- Implement refresh-token rotation with a revocation list so a stolen refresh token becomes useless after a single reuse.
- Add a simple search endpoint (title / description) with proper indexing — full-text search if your database supports it.

Details

You are required to provide the following as part of your submission:

- A detailed API specification with the expected request and response formats.
- A data model and database schema for users, events, bookings, and any auxiliary tables.
- A written note on how you prevent overbooking under concurrent requests, and what happens when two people try to grab the last seat at the same time.
- A plan for handling conflicts and error handling for invalid requests.
- Documentation for the API, including usage examples and error codes.
- Test cases and expected outcomes for both valid and invalid requests — at least one test should cover the concurrent-booking race.
- Guidelines for security measures, such as authentication, authorization, password hashing, and secret management.
- Instructions for deploying the backend service on a server (a Dockerfile and a short README section is enough).

Submission Guidelines:

Submit your project by **05th October 2026** in the provided form. Optional: Provide a link to your Git repository, a live deployment URL (Render, Railway, Fly.io, and Vercel all have free tiers that work), and any additional instructions if necessary.

Submission Form: Click here

Evaluation:

Your submission will be evaluated on the following criteria:

- Compliance with the requirements and specifications outlined in this document.
- Correctness under concurrency — does your booking flow actually hold up when hit in parallel?
- Quality and readability of code, including adherence to best practices and standards.
- Quality and completeness of documentation, including API specifications and usage examples.
- Accuracy and completeness of test cases and expected outcomes.
- Attention to security measures, such as authentication, authorization, and input validation.
- Ease of deployment and scalability of the service.
- Thoughtful attempts at the bonus challenges.

Resources:

You are free to use any language and framework you like (Node/Express, Django, FastAPI, Rails, Go, Spring — all fair game). Here are some good free resources to get you started:

The Odin Project

A free full-stack curriculum with a strong Node.js / Ruby on Rails backend track. Great if you're starting from zero. theodinproject.com

FastAPI — official tutorial

Modern async Python framework with JWT auth, dependency injection, Pydantic validation, and auto-generated OpenAPI docs baked in. Excellent official docs. fastapi.tiangolo.com/tutorial

Node.js + Express + Prisma

A step-by-step guide to building a REST API with modern Node tooling and a type-safe ORM. Prisma REST API guide

Django REST Framework tutorial

The go-to official tutorial for building REST APIs in Python with authentication, permissions, viewsets, and serializers. django-rest-framework.org/tutorial

PostgreSQL for backend developers

A practical, comprehensive PostgreSQL series covering everything from joins to transactions to indexes. Programming with Mosh – PostgreSQL tutorial

Redis in one video

Short, high-signal intro to caching, rate limiting, and pub/sub with Redis. Useful for the bonus tasks. Fireship – Redis in 100 seconds (+ full course link)

REST API design & OpenAPI

Microsoft's REST API design guide is a great, opinionated read on how public APIs should behave. Pair it with the OpenAPI spec docs when you start writing your Swagger file. Microsoft REST API guidelines | OpenAPI specification

JWT authentication — how it actually works

A short read that explains what's inside a JWT, why refresh tokens exist, and the common ways people get auth wrong. jwt.io Introduction

Concurrency in databases (row locks & transactions)

Directly relevant to the overbooking problem you'll hit. Read the transaction isolation and row-level locking sections. PostgreSQL docs — Concurrency Control

Docker for backend deployment

Once your service works locally, containerize it and deploy. This tutorial covers writing a Dockerfile, docker-compose, and pushing to production. Docker — Getting started

Good luck!
Best regards,
Web Cluster,
Team DCS